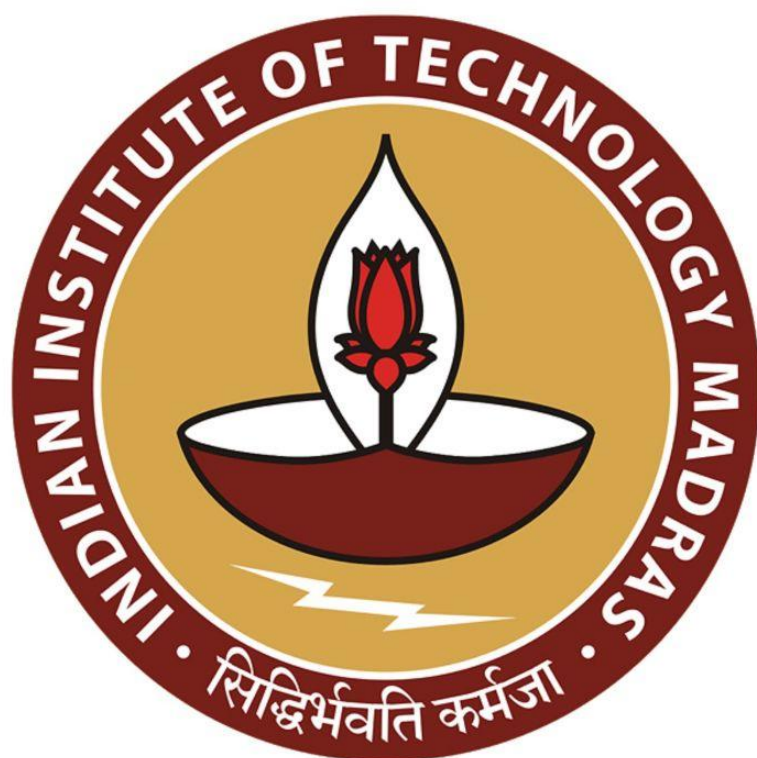




IIT Madras

ONLINE DEGREE

Machine Learning Techniques
Professor Arun RajKumar
Department of Computer Science and Engineering
Indian Institute of Technology, Madras
Decision Tree Algorithm

(Refer Slide Time: 0:20)

ALGORITHM - DECISION TREE

- DISCRETIZE each feature in $[min, max]$ range
- Pick the Question that has the largest Information gain.

Repeat the procedure for D_{yes}, D_{no}

```
graph TD
    A[" $f_k \leq \theta_k$ "] -- yes --> B[" $f_k \leq \theta_k$ "]
    A -- no --> C[" $f_k \leq \theta_k$ "]
    B -- yes --> D[" "]
    B -- no --> E[" "]
    C -- yes --> F[" "]
    C -- no --> G[" "]
    style D fill:none,stroke:none
    style E fill:none,stroke:none
    style F fill:none,stroke:none
    style G fill:none,stroke:none
```

So, the algorithm for decision tree is as follows. Step one, we need to first decide how many questions we are going to ask. So, we are only saying given a feature and value we know how good is this feature value question for the given data set, but then we know how many features are there, so that is given to us in the data set but what about this value.

So, we need to somehow pick certain values for these features and the most natural thing you can do is you can discretize each feature in the $[min, max]$ range. Meaning if the height is the feature and height the minimum height in your data set is 100 and the maximum height in

your data set is 200 then you discretize between 100 and 200 into 10 partitions which means that you will, the set of questions that you are going to ask is the height < 100 , is the height < 110 , height < 120 ,... height < 200 , so those are the questions that you are going to get.

So, now what you do is you pick the question that has the highest information gain or the largest information gain where information gain is exactly how we have defined earlier.

Now, what you do well you have found a single question that divides your data set into two parts which is D yes and D no. So, now D yes is a new data set which I have censored yes to this original question D no is another data set which I censored no to this question. Well, now we can ask, repeat the procedure for D yes and D no.

So, repeat the procedure for D yes and D no. By which I mean you first ask the question let us say you found out that the most information giving question is this. So, some $f(k \leq \theta_k)$. Now, this divides your data set into two parts, yes and no, now for the yes data set you again find that question which gives the most information gain.

Of course, every, I mean every data point in this set is going to answer $f_k \leq \theta_k$. So, there is no point in asking a question which is greater than θ_k for the feature k , for this side, so we already know that. So, you can reduce the number of questions you are asking, you can prune certain questions which are anyway useless, any question where $f_k \leq \theta$ where θ is bigger than θ_k is useless for this side, so you can leave out those questions.

Among the remaining questions you still have to figure out which one gives you the most information gain and then let us say that is some f_l for the left hand side $f_l \leq \theta_l$ is the question that that has the best information gain and then that will divide it into two parts yes and no and maybe on the other side the question is $f_r \leq \theta_r$ and that will divide it into yes and no and so on. So, and you can keep, you can keep going. So, this is how we build the tree. So, now here is a method where we can greedily keep building the tree as far as we want it to go. The question is how far do we build this tree.

(Refer Slide Time: 4:21)

POINTS

- Can stop growing a tree if a node becomes "SUFFICIENTLY" pure.
- DEPTH of the tree is a hyperparameter
- There are alternate measures for "goodness" of a question
→ GINI INDEX

IIT Madras
B.S. Degree

So, some points that we need to remember here are the following while building the tree some points. The first point is you can stop growing a tree if a node becomes sufficiently smooth, sufficiently pure rather. By which I mean that let us say you do not want complete purity but then if you keep asking this question that some point you will become like maybe your p is a bit either too small maybe it is between 0 to 0.1 or it is too large between 0.9 to 0.1 which means that it is your fraction of 1's is more than 90 percent or less than 10 percent in which case you are kind of saying that I do not have to grow this tree further.

At this point if p is fraction of 1's is greater than ninety percent then I will just make this a leaf node and say that the label is 1. If it is less than 10 percent then I will say that the label is 0. So, you can decide that threshold heuristically that is one thing to do. The other thing is that well even if you decide the threshold, your depth of the tree might actually be too large, so you maybe have, you may have to ask a lot of questions before you get to the sufficiently pure setup itself.

So, one way to, and what might happen in the process is that we will see when we look at the decision boundaries that when you try to make it more and more pure you will probably start fitting the noise. So, as how k was a parameter in k nearest neighbour which determines how many neighbours I should ask and which determined how smooth my boundary, decision boundary was,

Here the corresponding parameter would be the depth, so you need, you can, depth of the tree which is the number of questions I should ask before I answer 1 or -1, or 1 or 0 for a test data

point so that is the depth it is the, it is the you can think of it as the longest path from the root to a leaf node.

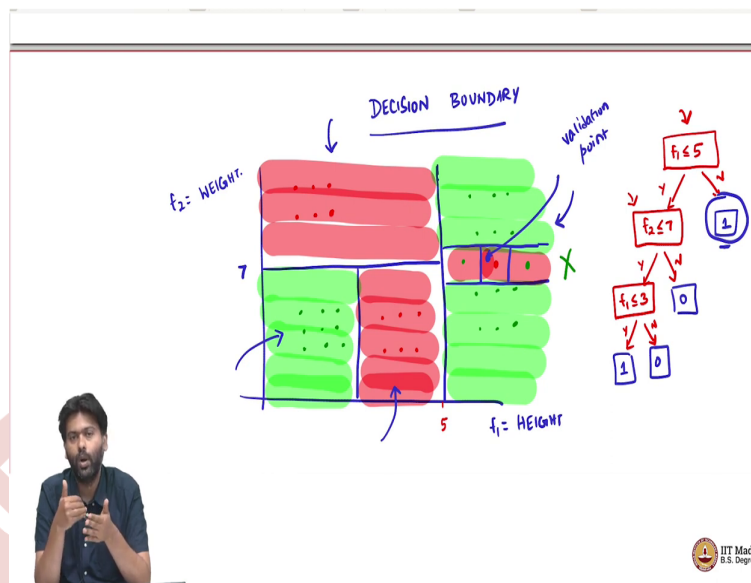
So, is a hyper parameter you can treat this as a hyper parameter which means that you will not grow the tree beyond this depth. Let us say if the tree becomes, if a node becomes sufficiently smooth even before the depth then you stop it, but then on one side it might become smooth and the other side it may not become smooth but even on the other side you would not let it grow forever, you will stop it at some depth, you will cut the tree off at the some depth.

And now for cutting it at different depths you can see how it performs on a validation set and then you cut it at the depth where you get the highest accuracy on the validation set, which means that you cross validate for this depth as a parameter. So, that is one other thing that one can do. So, these are two points to keep in mind while you are growing this tree.

The other point that I wanted to mention is there are alternate measures for impurity, measures for goodness of a question. Entropy is just one method that we, one popular measure of goodness, alternate measure of goodness for example that is called something called as a Gini Index which is another popularly used measure for goodness of picking goodness.

So, these all have different motivations but for our purposes entropy is very popularly used and so is Gini Index. So, in general we can think of any reasonable function which has this property that it becomes 0, when p is 0 or p is 1 and then it peaks at p is 0.5 and it has this symmetry property is a good it is a reasonably good measure of impurity. So, these are some points about how to build the street itself.

(Refer Slide Time: 8:35)



Now let us quickly talk about what comes out of this tree. So, what kind of decision boundary do we get. So, the decision boundaries, how are they going to look at for the decision tree, so let us talk about that. So, maybe I will go to the next page. Decision boundaries, so for this I will take an example where you just have two features let us say feature 1 is your height and feature 2 let us say is your weight.

And now you have a data set which, from which you built a tree which is, which looks like this let us say. So, $f_1 \leq 5$ then yes, no if it is no then you say 1, if it is yes then you ask $f_2 \leq 7$, if it is yes no, if it is no then you say 0, maybe this is the tree that you got.

So, this is just an example tree again you can ask $f_1 \leq 3$, if it is yes then you say 1, otherwise you say no. So, something like this. Well, this is the tree and what was the data set that we had well I should also mention the data set maybe your data set is something like this, maybe you have a bunch of points here, maybe you have a bunch of points here, and maybe you have an outlier here.

So, now what is happening here is what the first question that you might ask is $f_1 \leq 5$ maybe this is the 0.5 which means that gives us the separation like this. So, we are separating into two parts $f_1 < 5$ or $f_1 > 5$, if $f_1 > 5$ then we are already deciding it is class 1, so in this case we have already decided that it is class 1, let us say class 1 is the green class, so the whole part we are saying it is green class.

Now, if $f_1 < 5$ then we look at this data set the perhaps the question to ask now is f_2 the weight feature is it, sorry for that, f_2 the weight feature is it less than some value 7, maybe 7

is here. So, if it is not less than 7 which means if it is greater than 7 then we can perhaps decide that this is the red region, so that is the second, that is the second question that we are asking.

So, we first asked $f_1 < 5$ then we are asking $f_2 \leq 7$. Well, now if $f_2 \leq 7$ those data points which satisfy this are here and among those now we are asking again we might have to ask question about height. So, the same feature might repeat.

Of course, you will not ask a question about the feature with a greater value, you will never ask height great, height ≤ 6 for example, because you have already asked less than or equal to 5. So, you might ask height ≤ 3 though, so that is what we will ask here and that might divide it into two parts.

As you can see it might divide it into two parts and one side it is, on one side it is just green, on the other side it is, on this side is green, on the other side it is red. So, this is how your decision surface is going to be. Now, if I had cross-validated carefully then this is what my tree would have been, on the.

Now, what you observe here is that this node, let me use a different color, this node is a pure node, this node is a pure node, this partition is a pure partition, this is not a pure partition. So, because there is still this, so there was this red points that still, it got erased, but then there is still this red point.

So, but our cross validation would tell us that well this is just an outlier in some sense, we should not really be bothering about that. But if you are not cross-validated, but if you wanted complete purity of each nodes so then you would not stop this partition, what you would have done is you would have done asked more questions maybe question like this which would have divided into two parts, maybe another question like this which would have divided it again into two parts.

You would have kind of asked these questions about these features to box this guy separately and then you would have said well this box gets a red color which might be overfitting. So, basically what you are doing is that you are not stopping at this point here, you are not stopping at this point here, but then you are growing the tree on the right hand side. You are asking more questions about feature 2 here and then cutting it into smaller pieces.

Now, that might lead to overfitting, so you might ask questions like this and so on. You might somehow make this area very very pure if you will. So, maybe if there were green points

here, so then you would ask more questions to kind of surround this. Now, as you can see what is happening is every question is dividing our space by a line which is parallel to one of the axes.

So, it is drawing axis parallel lines to divide our regions into two parts, every question is doing that. A feature less than some value means that anybody, I mean basically that in that feature you are cutting it into two parts by drawing a line, so that is what is happening, that is what a decision tree is doing it is cutting a region into rectangles.

Now, you can imagine that no matter how complicated our +1's and -1's, our reds and blues, reds and green points are intermingled with each other you can still draw small enough rectangles around these boxes such that they will become completely pure. Of course, unless the same data point has both +1 and -1, +1 and 0 labels in our data set which let us say assume does not happen.

Now, as long as the points are all unique in our data set or in other words the same data point does not get both the labels, you can kind of close in each data point using sufficiently small rectangles. Which means that if you grow the tree long enough it is eventually going to become completely pure, all the nodes are going to become completely pure.

Because the number of data points that is falling on the nodes are getting reduced as you ask more and more questions and at some point it will just be one data point in the worst case and that is by definition by default a pure node, but that is a bad idea. So, just by seeking purity what we might do is fit the noise.

So, like how you your k being too small was fitting the noise in a k nearest neighbour, it made the decision boundary very non-smooth, here by growing the tree to large depth you will have these small holes in some sense. So, now to avoid this then we do this cross validation which will kind of say that well this is a bad idea, do not do this well because this is noise which means that in your test data if you had points in this region, now this classifier would actually give incorrectly for points in this region.

So, for example, maybe this was a test point, I will use a different color to indicate that it is a test point, maybe this was a test point let us say or rather a validation point because we do not have a test set in our hand. So, maybe this was a validation set point and what might happen is that in your validation set this complicated decision tree is going to predict incorrectly for

this point, whereas a more smoother decision tree where you had cross-validated for the height would not really predict incorrectly and so such a decision tree would be preferred.

So, which means you would kind of balance out the smoothness sufficiently such that you do well on the test data, so that is the hope by which you hopefully by cross validating the height you will capture the structure in the data in a much better way. So, this way of doing things is the method of decision trees which is a very very popular method and it is popular because the main reason why it is popular is because it is very interpretable.

So, now if I ask give a new data point and ask well the algorithm predicts makes a prediction as to +1. Now, let us say you are, let us say you are a doctor who is trying to use a decision tree to see if a particular medicine has to be prescribed for a patient or not or if a particular x-ray is I mean if maybe you look at some features and then say that if a particular person has a particular disease or not.

So, maybe not an x-ray but then you look at blood pressure, temperature, whatever it is, you look at a bunch of vital parameters and then you want to decide whether the person has a particular disease or not. Now, if your system says well this person has that disease. Now, as a doctor you will be worried why is the system saying that because you have to make some I mean you may have to do something based on this decision that the algorithm is telling you.

Now, as a doctor you want an explanation as to why a particular decision was made for a particular data point. Now, decision trees are very good at giving you that explanation, it will tell you that, hey this person was predicted to have a particular disease because his temperature was less than or was greater than this much, his blood pressure was less than this much and so on and so forth.

Of course, you should also imagine that you cannot give you, give the doctor a list of 100 different possible conditions. So, which means that if your tree height is too large then that is also perhaps not very explainable. So, you want shorter trees which are good for explaining and hopefully in most real world data sets you will not have to ask more than 10 questions or so which is also humanly interpretable.

So, if you explain it to a doctor perhaps the doctor is convinced, okay, so these are the parameters which the machine looked at to make this decision and of course the doctor can use his or her expertise to make further inferences, so that is, that comes from domain knowledge.

But as an algorithm this is very interpretable and we also have derived a model out of the data. Once this tree is there I can throw I do not care about the data points anymore. So, and so this is a good algorithm as well. So, at this point what we have seen so far in binary classification are two algorithms, one, two simple algorithms, one is k nearest neighbour and now we have looked at decision trees.

But both these algorithms are in some sense they are good algorithms at least decision tree is a good algorithm because it gives you a model but still it does not necessarily make any assumptions about the data at all. So, any data you give the algorithm will try to give you an answer which is good.

But now if we can make some reasonable assumptions about data then we can perhaps develop more algorithms which are better suited for the assumptions that we are making about the data. So, now what kind of assumptions we should make about data, what does it mean to say we are making assumptions about data, so and when we whenever we talk about assumptions there should be some sense of understanding the structure in the data and understanding the noise in the data, which means that somehow when we want to make assumptions about noise we should reason about probabilities.

So, far probability did not come into the picture at all when we talked about any of these algorithms. So, what we will see next is slightly more principled way of making assumptions about how the data itself is generated and that will give us two fascinating ways of modeling machine learning data generation.

And each of these space will give us a host of different algorithms and that is what we will see next where we will start looking at more probabilistic ways to model data generation and see what are their corresponding algorithms that we can do for binary classification, that is what we will start looking at from the next step. Thank you.